Branch Coverage Report

Project: Voice Assistant for Multimedia Controls - Team 3 **Prepared for:** Dhanush S S, Integration Lead **Execution date:** 07 November 2026, 01:50 IST (UTC+05:30) **Application source commit:** `91d5a74d815a2daff04afd896e4f135877d552b9` **Test mode:** author-run automated host tests; independent acceptance pending.

The course testing window shown in the brief is **2-6 November**. These tests were actually run on **7 November 2026 IST**; results are not backdated. UTC timestamps may fall on 6 November. Report date and evidence date are distinct from the course deadline.

Measured results

Scope	Covered branch outcomes / measured outcomes	Result
12 portable C++ implementation files	**457 / 518**	**88.22%**
Six optional Python prototype files	**14 / 36**	**38.89%**

Uncovered outcomes: **61 C++**, **22 Python**. Python additionally reports **4 partially covered branch points**. Its partial-branch count is not the number of missing outcomes. A branch outcome is a measured direction of a decision, not a test case or an independently accepted requirement.

LLVM measures instrumented source decision outcomes, including relevant short-circuit conditions. Python coverage.py measures its source branch arcs. They use different models; no pooled percentage is meaningful. **MC/DC was not enabled, and no MC/DC or complete-path-coverage claim is made.**

Explicit C++ denominator

All **12 existing portable implementation `.cpp` files** under `firmware/lib/` are compiled and included, including files not directly referenced by a suite. Only those `.cpp` mappings enter the percentages. **Headers/inline-only logic** (including `reset_button.h`), `firmware/src` hardware adapters/entry point, examples, tests, generated code, Unity, OpenSSL/framework internals and tooling are excluded. ESP32-only conditional code (notably mbedTLS) is not compiled in this host run. Unimplemented features have no code denominator and are not implicitly "covered". This is not whole-product or target-firmware coverage.

The scope inventory lists compiled files and excluded headers. The BOOT detector has six passing tests but no percentage in this measurement. Report it as tested, not as measured coverage.

C++ file results

Portable implementation file	Covered / branch outcomes	Branch coverage
asr_client/command_gate.cpp	13 / 14	92.86%
asr_client/command_matcher.cpp	110 / 114	96.49%
ble_hid/ble_hid.cpp	23 / 30	76.67%
command_map/command_map.cpp	125 / 126	99.21%
connectivity/authenticated_pairing.cpp	16 / 18	88.89%

Portable implementation file	Covered / branch outcomes	Branch coverage
connectivity/ble_discovery.cpp	10 / 12	83.33%
connectivity/credential_vault.cpp	73 / 100	73.00%
connectivity/factory_reset.cpp	6 / 6	100.00%
connectivity/wifi_provisioning.cpp	31 / 36	86.11%
diagnostics_ota/diagnostics_ota.cpp	15 / 22	68.18%
feedback/feedback.cpp	31 / 36	86.11%
pipeline/command_dispatch.cpp	4 / 4	100.00%

Optional Python file results

Prototype file	Covered / branch outcomes	Branch coverage
command_classifier.py	2 / 2	100.00%
command_mapper.py	0 / 0	N/A - no measured branches
main.py	7 / 10	70.00%
media_controller.py	5 / 12	41.67%
receive_audio.py	0 / 10	0.00%
speech_to_text.py	0 / 2	0.00%

Files with no measured branches are N/A, not automatically 100% branch-tested.

Remaining branch work

• Credential vault: 27 missing outcomes; inspect input, crypto/allocation and storage-failure decisions. Native results exclude the ESP32 mbedTLS path.

• BLE sender: 7 missing outcomes, including default/stub and edge-case paths.

• Diagnostics: 7 missing outcomes; inspect configured log edge cases.

• Other missing C++ outcomes are shown individually in the annotated evidence.

• Python serial capture and transcription are wholly unexecuted. Add suitable protocol/error tests and separately record real serial/model tests.

• Real Windows key dispatch requires appropriate platform tests; mock-based mapping tests do not exercise the operating-system calls.

Allocation-failure/error branches need deliberate, meaningful fault injection; do not mark them covered because the normal path passed. No arbitrary branch threshold or new acceptance criterion is introduced by this report.

Scope and acceptance boundary

The approved project uses ESP32-WROOM and offline fixed-vocabulary recognition. No live cloud API, microphone, flash operation or Bluetooth/media action was performed by this report run. Passing host tests is not completion of the physical SRS/STP acceptance criteria. This report does not set RTM A/F statuses, close issues, complete INT-02, create a release tag or freeze development.

The locally revised SRS/SAD/STP/backlog implement Dhanush's offline redesign; some revisions are still unpublished and remote source text remains inconsistent. The exact source-document hashes are in source-baselines.json. Harsh/DOC-01 must reconcile and publish those baselines before final acceptance. Legacy `TC-ASR-*` IDs are retained; their names do not authorize cloud recognition.

Reproduction and evidence

From the repository root, with PlatformIO installed:

```
pio test -d firmware -e native
python3 -m pip install coverage==7.17.2
python3 scripts/measure_coverage.py --output /tmp/vamc-report-rerun
```

The coverage runner currently supports **macOS** with Xcode command-line tools, Homebrew `openssl@3` and Unity installed by the PlatformIO test run. It compiles the same existing C++ tests and library sources with Apple Clang/LLVM 21 at `-O0`, using `-fprofile-instr-generate -fcoverage-mapping`. Test/Unity code is not instrumented. All 12 portable `.cpp` objects are linked into every suite; the 15 raw profiles are merged once, and one binary provides their shared mappings. This avoids counting shared code 15 times. The runner then invokes Python coverage with `--branch`, including all six prototype source files and excluding its tests. It records tool versions, commands, source commit and suite counts.

- Run manifest and exact commands
- Coverage scope inventory
- PlatformIO native test output
- C++ coverage summary
- C++ annotated lines and branch counts
- C++ machine-readable coverage
- Python test output
- Python coverage summary
- Python machine-readable coverage
- Python annotated HTML

Method reference: Clang source-based coverage documentation. Execution evidence, rather than documentation examples, supplies every result here.